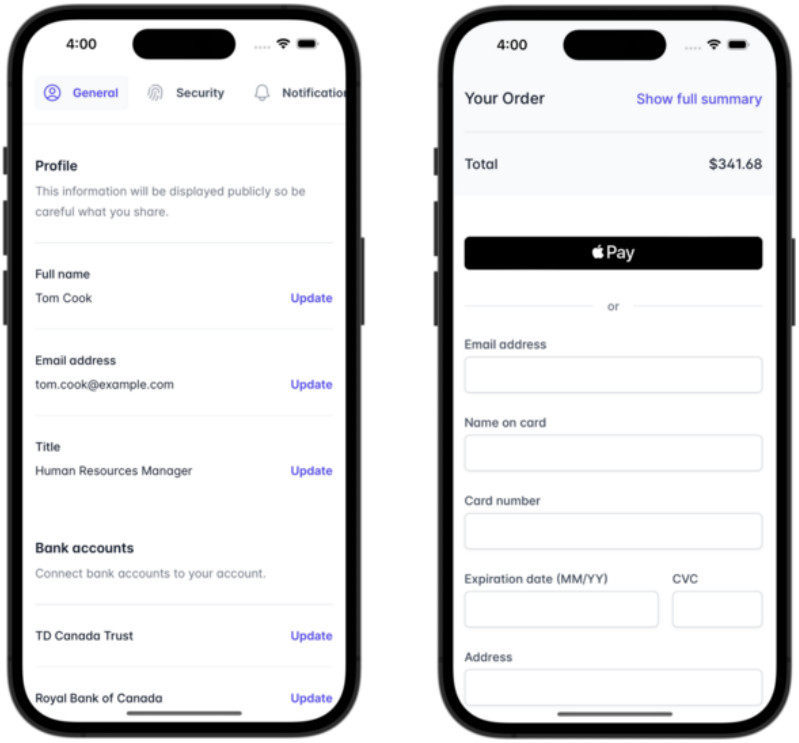
The following screenshot shows a settings screen on the left and a checkout flow on the right. These screens change frequently, for example, adding a new profile field or a way to use coupon codes when checking out. It's best to keep these web-based so they require less code to write and maintain.



Boring *CRUD-like operations* that aren't unique to your app's experience or product probably don't need to be native. Yes, they might be fun to experiment with. But the time and resources spent are most likely better served when working on critical workflows like native home screens and maps or interacting heavily with native APIs.

Lastly, rendering *a lot of dynamic content* is often quicker to build on the web. If we went native, a list of heterogeneous items, like a news feed, requires each item type to be implemented as its own native view. And each *new* item type requires new releases to the app stores. Leaving all this logic and rendering to the server helps ensure the apps won't block new features on the web.